children and, in turn, all elements in its subtree. Bottom: the same heap in its array representation.

But the improvement we get with trees comes to a price, of course. First, as with any data structure that uses pointers (lists, graphs, trees, etc...) we have a memory overhead in comparison to arrays. While with the latter we just need to reserve space for the data (plus maybe, depending on the implementation details, some constant space for pointers and the node structure itself), every tree node requires extra space for the pointers to its children and possibly to its parent.

Without getting too much into details, arrays also tend to be better at exploiting *memory locality*: all the elements in the array are contiguous in memory, and this means lower latency when reading them.

Table 2.3 shows how a heap matches a priority queue for its abstract part, and what are its concrete data model and invariants.

Table 2.3 Underlying components for a heap

Concrete Data Structure: Heap	

API	`Heap {`
	`  top() → element`
	`  peek()→ element`
	`  insert(element, priority)`
	`  remove(element)`
	`  update(element, newPriority)`
	`}`

Contract with client	The top element returned by the queue is always the element with